

Final Report

Developing Simulation Tools for Vascular Bloodflow Modelling using XLB

Muhammad-Abdullah Kashif-Khan

Submitted in accordance with the requirements for the degree of
MEng, BSc Computer Science with Artificial Intelligence

2024/25

COMP3931 Individual Project

The candidate confirms that the following have been submitted.

Items	Format	Recipient(s) and Date
Final Report	PDF file	Uploaded to Minerva (14/05/25)
Link to online code repository	URL	Sent to supervisor and assessor (14/05/25)

The candidate confirms that the work submitted is their own and the appropriate credit has been given where reference has been made to the work of others.

I understand that failure to attribute material which is obtained from another source may be considered as plagiarism.

(Signature of Student) Muhammad Kashif-Khan

Summary

Within a clinical setting, there is an increasing need to conduct aneurysm rupture risk assessments upon patients where aneurysms have been detected. Current methods of assessment using computational flow dynamics (CFD) via the finite element method (FEM) remain computationally expensive and limited in solutions for scalability and boosting performance.

Alternatives for modelling, namely the lattice Boltzmann method (LBM), provide promise into fast modelling techniques with GPU-accelerated architecture. The application of such tools requires precise physiologic and haemodynamic modelling, which includes simulating non-Newtonian blood flow, deriving wall shear stress as part of the risk assessment strategy, and modelling blood flow according to heart rate and blood pressure. While LBM has been explored in research, few studies address clinical-ready implementations with physiologic boundary conditions and GPU-optimized workflows.

To bridge this gap and enable clinical transition, we present the groundwork for developing tools for one such open-source LBM simulator, namely XLB, for a basic 2D cross-sectional model of an aneurysm inside the common carotid artery (CCA). We extend XLB with custom CSV-based inlet profiles, boundary conditions accounting for non-Newtonian behaviour using the Carreau–Yasuda model, and wall shear stress (WSS) computation which enables realistic physiologic and haemodynamic mechanisms, and data extraction processes. We evaluate the performance of these implementations in terms of accuracy and computational speed.

The implication for this project is vast. Successful 2D implementation enables scaling to 3D with minimal architectural changes, paving the way for GPU-cluster deployment and AI-driven risk prediction. Overall, this work not only has the potential to improve patient care but also serves as a foundation for vascular research (e.g., stent deployment within compromised vascular systems).

Acknowledgements

<The page should contain any acknowledgements to those who have assisted with your work. Where you have worked as part of a team, you should, where appropriate, reference to any contribution made by other to the project.>

Note that it is not acceptable to solicit assistance on ‘proof reading’ which is defined as the “the systematic checking and identification of errors in spelling, punctuation, grammar and sentence construction, formatting and layout in the text”; see

https://www.leeds.ac.uk/secretariat/documents/proof_reading_policy.pdf

Contents

1	Introduction and Background Research	1
1.1	Introduction and Motivation	1
1.2	Fundamentals of Fluid Dynamics & Blood Flow	2
1.2.1	The Navier-Stokes Equations & Fluid Flow	2
1.2.2	The Circulatory System & Aneurysm Formation	4
1.3	The Lattice-Boltzmann Method (LBM) & Its Application	4
1.4	GPU-Accelerated LBM Using XLB	5
1.5	Computational Blood Flow Modelling & Aneurysm Prediction	6
1.6	Literature Review & Existing Research	7
2	Methodology	10
2.1	Overview of the Simulation Approach	10
2.2	Simulation Setup & Execution Details	11
2.3	Aims and Objectives	14
2.4	Project Management, Version Control, & Testing	16
3	Implementation and Validation	18
3.1	Implementation Details	18
3.2	Implementation Challenges	20
3.3	Validation of Key Objectives	21
4	Results, Evaluation and Discussion	25
4.1	Evaluation	25
4.2	Conclusions	28
4.3	Ideas for Future Work	28
4.3.1	Further Additions to the 2D Model	29
4.3.2	Transition to 3D Modelling	29
4.3.3	Extension of Warp to Distributed Architectures	30
4.3.4	Incorporating ML Pipelines	30
4.3.5	Implementing Clinical and Research-Based Tools	30
	References	32
	Appendices	35
A	Self-appraisal	35
A.1	Critical self-evaluation	35
A.2	Personal reflection and lessons learned	36
A.3	Legal, social, ethical and professional issues	36
A.3.1	Legal issues	36
A.3.2	Social issues	37

A.3.3 Ethical issues	38
A.3.4 Professional issues	39
B External Material	41

Chapter 1

Introduction and Background Research

1.1 Introduction and Motivation

An aneurysm is a localised, abnormal bulge in the wall of a blood vessel, typically an artery, caused by a weakened vessel wall. If left untreated, it may enlarge and rupture, leading to life-threatening internal bleeding [1]. Unruptured intracranial aneurysms are present in approximately 3.2% of the general adult population [2], with an estimated annual rupture rate of 1.4% [3]. The physiological structure of a blood vessel and haemodynamics influence the initiation and progression of aneurysms, where high-velocity blood flow circulates around areas of weakened or curved vascular structure [4] [5]. Blood velocity can be modulated by systemic factors such as elevated blood pressure and heart rate [6], as well as fluid shear stress and turbulence.

Within clinical settings, MRI and CT scans are commonly used to infer key physiological parameters such as aneurysm size, shape, neck-to-dome ratio, location, and wall characteristics [7]. Haematological parameters local to the aneurysm site can further inform surgical decision-making [8]. While imaging offers anatomical insights, dynamic flow analysis relies on computational modelling, where current tools still face critical bottlenecks.

Currently many different solutions for simulating blood flow through aneurysms have been developed, but exist primarily in a research setting. an example is the work done by Hanieh Mirzaee et al, developing a 4D flow MRI data to construct a patient specific haemodynamic simulations using the Lattice-Boltzmann method. This study did not propose integration methods of such a model to a clinical environment, though validates the applicability for such a tool. [9].

In addition, tools established for simulating blood flow through vascular structures such as SimVascular, rely on a computationally expensive approach using the finite element method (FEM) with vascular meshing techniques [10]. The FEM technique provides limited parallelisation and scalability options [11], which may prove crucial when dealing with urgent, time-critical cases. This project aims to address the weaknesses of these other systems.

Lattice-Boltzmann methods (LBM) aim to address this computational inefficiency by providing a significant computational advantage over traditional CFD methods, resulting in faster, parallelisable, scalable computation while being efficient for complex geometries [12]. We aim to develop a fast, LBM-based blood flow simulation tool as a proof-of-concept for future developments in research and clinical settings, with capabilities in GPU acceleration and distributed computation, and compatibility with AI-driven risk prediction pipelines. This paper serves as the initial development stage, looking into developing the necessary tools required for realistic simulations for a basic 2D aneurysm, before further development.

1.2 Fundamentals of Fluid Dynamics & Blood Flow

1.2.1 The Navier-Stokes Equations & Fluid Flow

Developing an understanding of the underlying concepts of fluid flow will help to understand the requirements for developing the necessary tools for realistic simulation. The Navier-Stokes equations (NSE) are a set of equations dictating the behaviour of fluid flow

$$\rho \left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} \right) = -\nabla p + \mu \nabla^2 \mathbf{u} + \mathbf{f}_{\text{body}} + \mathbf{f}_{\text{wall}} \quad (1.1)$$

$$\nabla \cdot \mathbf{u} = 0 \quad (1.2)$$

Where ρ is the blood density ($\approx 1060 \text{ kg/m}^3$), μ is the dynamic viscosity of blood ($\approx 3.5 \times 10^{-3} \text{ Pa} \cdot \text{s}$ for Newtonian assumption), $\mathbf{u}$ representing the velocity field in vector form, p being pressure, $\mathbf{f}_{\text{body}}$ being body forces (e.g. gravity, often negligible in arteries), and a non-conventional addition to the formula, namely $\mathbf{f}_{\text{wall}}$ dictating interaction forces from arterial walls, which can include vessel elasticity [13]. While some studies (e.g., Failer et al.) neglect $\mathbf{f}_{\text{body}}$ and $\mathbf{f}_{\text{wall}}$ for generalized cardiovascular modeling, these forces are critical for compromised vessels—particularly with wall thinning and increased elasticity during aneurysm development [14] [15]. We retain these terms to accommodate factors beyond this paper’s scope but focus on the following:

Equation 1.1 can be deconstructed into different related force equations. The formulas $-\nabla p$, $\mu \nabla^2 \mathbf{u}$, are seen as the pressure gradient, and viscous/internal fluid frictional forces respectively. The pressure gradient drives fluid (i.e. blood) from high pressure to low pressure regions. These sum up to $\rho \left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} \right)$, which dictates the inertial forces acting within a fluid. We can deconstruct this term to dictate the localised acceleration (representing how velocity changes over time at a fixed point in space), and convective acceleration (relating to fluid’s velocity changes as it flows, due to narrowing of the vessel, bends, or branching) [16].

The nabla symbol ∇ is a vector differential operator used to compute spatial derivatives of scalar and vector fields—specifically the gradient, divergence, curl, and Laplacian. In the Navier–Stokes equations (NSE), ∇ represents partial differentiation in 2D or 3D space; for this project, we consider the 2D case [17]:

$$\nabla = \left(\frac{\partial}{\partial x}, \frac{\partial}{\partial y} \right)$$

Applying $\nabla \cdot \mathbf{u}$ yields the divergence of the velocity field, representing fluid expansion or compression at a given point. Setting this to zero, as in the continuity equation, implies incompressible flow—no net change in local volume—a common assumption for blood and other biological fluids [16]. The curl $\nabla \times \mathbf{u}$, meanwhile, measures local rotation or vorticity. While not sufficient alone to characterise turbulence, it serves as a diagnostic for identifying disturbed or circulating flow regions. Together, divergence and curl help characterise the fluid dynamics and validate conservation laws, as illustrated in Figure 1.1.

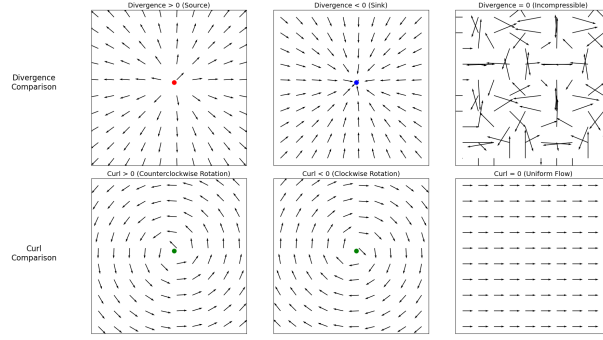


Figure 1.1: Comparison of divergence and curl using Flow pattern diagrams.

Viscosity (μ) is a measure of a fluid's resistance to deformation under shear, quantifying internal friction as adjacent fluid layers slide past each other. This relationship between shear stress and shear rate is expressed as [16]:

$$\tau = \mu \dot{\gamma} \quad (1.3)$$

In Newtonian fluids, μ remains constant, yielding a linear relationship between shear stress and shear rate. In contrast, many biological fluids like blood are non-Newtonian and exhibit shear-thinning behaviour, where viscosity decreases with increasing shear rate. While some studies dismiss this effect in large vessels, it becomes significant in smaller diameter vessels where shear gradients are higher [18].

Wall shear stress (WSS) is the tangential force per unit area exerted by blood flow on the vessel wall, influencing endothelial function, vascular remodelling, and the development of pathologies such as aneurysms. It is defined as:

$$\tau_w = \mu \left. \frac{\partial u}{\partial n} \right|_{\text{wall}} \quad (1.4)$$

where τ_w is the wall shear stress, μ is the dynamic viscosity, u is the velocity component parallel to the wall, and n is the normal direction. In disturbed flow regions such as aneurysms, abnormal WSS is linked to disease progression and rupture risk, making WSS estimation important for clinical assessment [19, 20].

Fluid flow is classified as laminar or turbulent based on particle behaviour. Laminar flow is smooth and orderly, occurring at low velocities or in small vessels, while turbulent flow is chaotic, with vortices and increased energy dissipation, typically arising in large vessels or where geometry disrupts flow. The Reynolds number (Re) quantifies this, defined as

$$\text{Re} = \frac{\rho u D}{\mu} \quad (1.5)$$

where ρ is fluid density, u is velocity, D is characteristic length, and μ is dynamic viscosity. Laminar flow generally occurs when $\text{Re} < 2000$, turbulent flow when $\text{Re} > 4000$, and values between represent transitional flow. Blood flow in healthy arteries is mostly laminar, but turbulence may occur in aneurysms, stenoses, or near bifurcations [21].

1.2.2 The Circulatory System & Aneurysm Formation

Blood circulates around the body according to the mechanical action of the heart. The amount of blood pumped around the body depends on cardiac output rate, which is influenced by factors such as blood pressure, heart rate, and myocardial contractility. When the heart contracts and relaxes, pulsatile blood flow circulates. This pulsing blood flow creates stress-strain cycles on the vascular structure of the body, particularly where vessels curve, narrow or branch. These stress-strain cycles can contribute to mechanical fatigue in the vessel walls over time, particularly in areas with turbulent or high-velocity flow, such as the coronary arteries, cerebral arteries, and aortic bifurcation. Over years, this repetitive mechanical stress may play a role in vascular remodelling, endothelial dysfunction, and the development of conditions like atherosclerosis or aneurysms. Increased cardiac output, whether due to hypertension, exertion, or underlying pathology, exacerbates these stress-strain cycles and accelerates vessel wall fatigue. This is reflected in the NSE pressure term $-\nabla p$, where higher pressure gradients directly increase wall stress [22].

Arteries are structurally adapted to withstand high wall shear stress (WSS) and cyclic deformation, with thicker walls composed of elastin and collagen. Veins, in contrast, operate at lower pressures and lack comparable structural reinforcement. Consequently, arteries possess a greater chance of developing abnormalities in the vessel structure due to localised stress concentrations at vessel bifurcations exceeding physiological thresholds. Turbulent flow and flow separation in these regions create both high wall shear stress zones, causing endothelial damage and vessel degradation, and low wall shear stress zones, causing inflammation and weakening vessel walls [4]. Clinically, aneurysms most frequently form at bifurcations (e.g., 85% of cerebral aneurysms occur in the Circle of Willis [23]), where flow separation creates regions of oscillatory WSS. These haemodynamic factors are poorly captured by static imaging, necessitating computational tools like LBM to simulate dynamic stress distributions [24].

Additionally, physiological factors such as vasoconstriction and vasodilation influence local pressure and flow velocities, altering stress distributions along vessel walls. Pharmacological agents, such as blood-thinning medications, can modify blood viscosity, directly impacting WSS and overall haemodynamic stability. These influences further support the need for time-resolved simulation frameworks capable of incorporating variable boundary and flow conditions in a patient-specific manner.

1.3 The Lattice-Boltzmann Method (LBM) & Its Application

The Lattice-Boltzmann Method (LBM) is a mesoscopic scheme for simulating fluid flow by approximating the Boltzmann transport equation, providing a statistical view of particle dynamics. Instead of solving the Navier-Stokes equations directly, LBM models fluid behaviour through two main steps: collision, where particles relax toward equilibrium, and streaming, where they move to adjacent lattice nodes. Its simple, local operations make LBM well-suited for efficient, highly parallel computation, especially on GPUs [25].

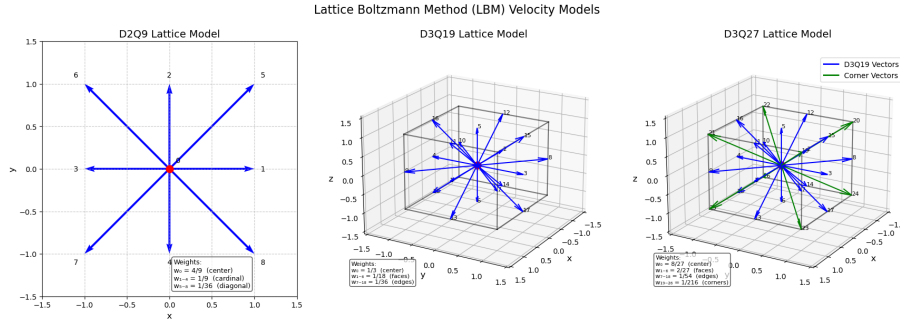


Figure 1.2: LBM velocity models for 2D (left) and 3D simulations (centre and right)

LBM operates at both the mesoscopic scale, capturing the statistical behaviour of particle ensembles, and the macroscopic scale, where fluid properties such as velocity and pressure are recovered by taking moments of the distribution function. A common simplification of this model is the Bhatnagar–Gross–Krook (BGK) approximation, which introduces a single relaxation time to model the return to equilibrium. The BGK method is particularly effective for simulating low Reynolds number, incompressible flows. A more detailed explanation of the BGK approximation, including its decomposition and relevance, is provided later in Section 2.

Simulations require the definition of a discretised domain space, which can be arbitrarily shaped to accommodate complex geometries such as vascular networks. The fluid domain is populated by lattice nodes, each associated with a finite set of discrete velocity directions defined by the chosen velocity model. In this work, the D2Q9 model is used, representing a two-dimensional grid with nine velocity directions, including a rest state. Each direction is assigned a weight that ensures isotropy and accurate macroscopic recovery. For three-dimensional simulations, higher-order velocity models such as D3Q19 and D3Q27 are commonly employed. These models are illustrated in Figure 1.2, which shows the configuration of lattice directions for both 2D and 3D cases. Boundary conditions must also be defined for the inlet, outlet, and walls, controlling how particles reflect or enter/exit the domain. Selecting appropriate boundary schemes is critical to achieving realistic flow behaviour.

Although LBM can be less accurate than finite element or finite volume methods in certain regimes due to compressibility errors or lattice resolution constraints, its exceptional performance on parallel hardware makes it well-suited for real-time or large-scale simulations. This efficiency is especially important in clinical applications where speed, interactivity, and physiological realism are prioritised over perfect accuracy.

1.4 GPU-Accelerated LBM Using XLB

XLB is a Python-based GPU-accelerated publicly available LBM solver designed for fast fluid dynamics simulations. It leverages parallel processing backends such as the JAX and Nvidia Warp libraries to allow efficient parallelisation for the LBM solver [26]. At the time of writing, XLB is in the development stage with version 0.2.1 available for use.

XLB offers distinct advantages over other LBM solvers, notably support for distributed architectures and seamless AI integration via its JAX backend. JAX, developed by Google, enables high-performance numerical computing through just-in-time compilation and automatic differentiation, making it well-suited for embedding machine learning into scientific workflows. Warp, developed by NVIDIA, is a CUDA-accelerated framework offering low-level GPU control through kernel-style operations, enabling fine-grained optimisation with minimal overhead. These capabilities, wrapped in a Python interface, align with clinical requirements by enabling scalable, GPU-accelerated simulations. Although not as performance-optimised as C++ frameworks like OpenLB, XLB excels in usability and rapid prototyping [27].

XLB remains under active development and lacks several essential features for this project, including support for graph-based variable velocity profiles, non-Newtonian fluid modelling, and WSS extraction. However, it provides a functional NSE solver, core boundary conditions, and example implementations that can be adapted with appropriate data formatting. The rationale for choosing XLB, despite these limitations, is further discussed in Section 1.6, where it is evaluated against alternative LBM frameworks in terms of performance, extensibility, and clinical suitability.

1.5 Computational Blood Flow Modelling & Aneurysm Prediction

Simulating aneurysms is challenging. As shown by the NSE, there are several factors and variables required to produce realistic results. A combination of biological, mechanical, and physiological factors are in play when attempting to account for a whole system, often with these factors intertwined as part of a complex system. Generalisation without loss of realism is essential for developing high level simulation and analysis tools. Such simulation tools need to account for and capture complex blood vessel geometries, the blood’s non-Newtonian properties, wall compliance & elasticity. Simulations must also account for realistic scenarios people experience and develop margins for error to account for external influences that may boost aneurysm rupture chances, such as heavy impacts to the body and sneezing, each of which can amplify local pressure in the diseased vessel.

Currently, risk assessment strategies rely on patient details and compromised vessel anatomy, extracting features such as aneurysm size, shape, location, daughter sac development, and patient-specific risk factors like hypertension, smoking history, and age [28]. As a result, studies attempt to create machine learning (ML) methods, accounting for clinical and demographic features, image based models, morphological structure, and haemodynamics. A combination of all such features proves to show the highest performance in terms of accuracy and effectiveness, however haemodynamics and WSS analysis is not widely adopted for use in ML, particularly due to the cost in calculating and extracting this data.

1.6 Literature Review & Existing Research

There are multiple computational approaches for NSE simulation, each offering distinct benefits depending on the application. In this project, the focus is on selecting a method suitable for clinical environments, where speed, reliability, and usability are critical. Specifically, the simulation tool must deliver near real-time performance, with clinicians expecting results within minutes. It should also provide accurate and interpretable outputs, supported by tools for clinical analysis, while offering a high degree of automation and integration to minimise bureaucratic overhead and training requirements [29, 30]. In addition, the system must be easy to maintain and accessible for external scrutiny, allowing researchers and clinicians to inspect, adapt, or validate the tool for use in their own testing and research environments.

Academic literature predominantly revolves around 2 different computational methods for resolving simulation space, namely the finite element method (FEM) and LBM. FEM discretizes complex geometries into finite elements, solving simplified governing equations to approximate system behaviour. While highly accurate, it is computationally expensive due to meshing complexity and iterative solvers, particularly for vascular flows [25]. In contrast, LBM offers a highly parallelisable framework that delivers significantly faster performance, making it better suited for time-sensitive clinical scenarios. Although this speed comes at the cost of reduced accuracy in some cases, the trade-off is acceptable when prioritising rapid diagnostics and real-time simulation support in urgent medical contexts.

There are many LBM solvers openly accessible to developers, including C++-based libraries such as Palabos and OpenLB, and Python-based alternatives like LettuceCFD and XLB. While some C++ implementations offer Python frontends to improve accessibility, these typically act as wrappers over complex C++ backends. This limits their maintainability and ease of scrutiny, especially for researchers seeking to understand, modify, or extend the codebase directly. In contrast, a fully Python-based framework facilitates easier inspection, faster prototyping, and greater integration with scientific tooling and ML pipelines, key priorities in a research and clinical setting.

To assess practical performance trade-offs, we conducted benchmark tests comparing LettuceCFD and both JAX- and Warp-based backends in XLB, measuring throughput in Million Lattice Updates Per Second (MLUPS). Tests were run on a consumer-grade system with an NVIDIA RTX 3060 GPU (6GB VRAM), using the standard 3D lid-driven cavity flow (XLB) and 2D Taylor-Green vortex (Lettuce) simulations. The results, presented in Table 1.1, show that XLB using Warp vastly outperforms both JAX and Lettuce backends, demonstrating significantly higher throughput and memory efficiency. These findings support the use of Warp for computational performance, particularly in time-constrained clinical applications.

Based on these insights, we select XLB for its Python-native interface, adaptable architecture, and support for Warp, combining ease of use with scalable GPU performance. Its open development base (led by Autodesk Research) and rich example set further support the goals of this project, particularly in constructing 2D aneurysm simulations and extending the library's

capabilities for clinical research.

Library	Backend	Benchmark Flow	Domain	MLUPS (avg)	Key Notes
LettuceCFD	PyTorch	2D Taylor-Green Vortex	1024×1024	326	Fully periodic domain using D2Q9 stencil
XLB	JAX	3D Lid-Driven Cavity	128^3	216	Exceeded VRAM limit on 256^3 domain; memory inefficiencies at scale
XLB	Warp	3D Lid-Driven Cavity	128^3	1757	Fastest runtime; low-level GPU control via CUDA wrapper with minimal overhead

Table 1.1: Performance comparison of LBM solvers using FP32 precision (single precision).

Blood tends to exhibit non-Newtonian rheology, complicating LBM simulation efforts due to the BGK model’s single relaxation time calculated by a fixed viscosity. As a result, the BGK collision operator struggles to capture the effects of non-Newtonian fluids with shear-dependent viscosity, which if implemented, yields spatially varying relaxation times. This mismatch causes numerical instability and reduced accuracy, especially under high shear or in off-grid geometries [31]. Other collision operators exist, such as the multiple-relaxation-time (MRT) and two-relaxation-time (TRT) that are able to improve the simulation stability, however these more complicated operators are more computationally expensive and challenging to implement within the context of XLB.

Studies tend to leave out the role of biomechanical systems influenced by haemodynamic factors, such as local WSS, possible effects of inflammation and immune responses, and flow patterns and recirculation zones for developing criteria and ML methods deployable in a clinical setting. This is primarily due to data scarcity and measurement difficulty, computational burden and model complexity, and a lack of standardised, clinically validated endpoints. We hint at the possibility for simulation tools such as XLB to incorporate ML-based methods to promote realism, possibly via the development of a physics informed neural network (PINN) inspired by biomechanical systems [32], however this falls outside the scope of the project, and would require extensive further study into how this can be achieved. Because of the Pythonic codebase, it may also be possible to extract haematological features and pipeline the results directly into an ML tool.

A major contributing factor and motivation for this type of project is the significant lack of adoption of robust aneurysm rupture predictors. Xiang [30] seems to have the closest result to having a full scale aneurysm predictor and simulation tool, but suffers from a lack of adoption, links to further developments in the software or plans to introduce the tool to a wider set of clinics and hospitals. It would be intriguing to understand the reasons why this didn’t follow through despite positive responses from the test centres. As a USA based research paper, this may be down to the requirements stipulated by the FDA, requiring approval before considering widespread adoption.

Overall, this project serves as a unifying force to combine the studies applied for different specific problems, such as looking at aneurysms in specific locations. This paper attempts to generalise these pipelines, creating tools that can be applied to any collection of vascular modelling, given correct input parameters and meshing structures for future 3D modelling. In general, we want to unify existing research into a package deployable for use in research and

clinical applications, serving as a platform for reducing LBM development time and cost for all. The project aims to resolve a standing data science and software engineering problem by providing the methods to extract these haematological features, and provides ambitious ideas to tackle widespread adoption of these vital clinical tools.

Chapter 2

Methodology

2.1 Overview of the Simulation Approach

We want to develop generalised tools that can be used to simulate vascular fluid flow using the LBM approach. To do this, we detail the architecture of XLB with a high-level take on execution flow and data computation.

The simulation space is intended to take place on a 2D domain space, aiming to validate the developed tools and functions to different test domains before further development into 3D space, which we do not cover. To start the drive to model vascular flow accuracy, we focus the development on physiological flow profiles, namely trying to reflect the pulsatile flow the heart induces across the body, and reflecting non-Newtonian shear-thinning fluids within the simulation space to reflect the terms $-\nabla p$ and $\mu \nabla^2 \mathbf{u}$, in the NSE respectively (see Eq 1.1). We also need to add capabilities to calculate and extract the WSS along the walls of the simulation domain.

We start by analysing Figure 2.1, representing the execution flow of XLB against a standard simulation run available on GitHub. The entirety of the simulation, including the creation of the domain space, boundary condition (BC) labelling, and post-processing methods are encompassed in 1 overall class. We identify 2 major areas required to create and run the simulation space. Within these setup and execution calls, the example code defines functions that outline a step-by-step process that can be interpreted to create our own simulations. For our purposes, we focus on the development of a new BC, incorporation of non-Newtonian physics to the collision operator and develop methods in the post-processing stage to calculate the WSS.

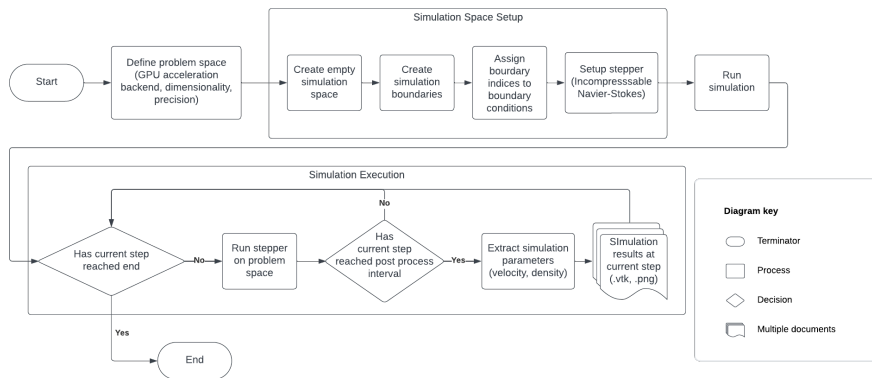


Figure 2.1: Generalised XLB Execution flow chart

2.2 Simulation Setup & Execution Details

As part of this paper, we present the notion of fluid flow simulation being conducted on a rudimentary 2D cross-section construction of the common carotid artery (CCA) with an aneurysm. We address the design requirements in order of the flow diagram shown by Figure 2.1.

The Lattice Boltzmann Method operates in lattice units, which are inherently non-dimensional and arbitrary. To ensure physical relevance and realism in our simulations, we must define a consistent mapping between lattice units and physical units Δx and timestep Δt . This requires careful selection of reference quantities such as characteristic length, velocity, and time scale, so that key physical parameters like the Reynolds number remain consistent with physiological conditions. Accordingly, we construct the simulation domain and parametrise the model in a way that reflects the appropriate physical dimensions of the vascular system being studied.

Larger values of Δx and Δy result in fewer cells and coarser resolution, which can limit the accuracy of simulation outputs. Conversely, smaller values significantly increase grid size and memory requirements, as the number of lattice nodes grows quadratically (or cubically in 3D). Additionally, finer spatial resolution demands smaller Δt values to maintain numerical stability, which increases the total number of timesteps and overall computational cost. These trade-offs were experimentally examined to maximise both resolution and timestep while keeping simulations numerically stable and within system constraints, namely, not exceeding the 6GB VRAM available on the test laptop.

To begin the implementation requirements, the domain size, analogous to a 2D array of arbitrary dimensions X and Y , must be selected to achieve sufficient precision without compromise of computational efficiency. A domain too small obstructs the mesoscopic properties on which the LBM depends, potentially compromising physical accuracy. Conversely, a larger domain significantly increases computational cost scaling with complexity as $\mathcal{O}(X \times Y)$. As such, domain selection plays a critical role in ensuring both the physical fidelity and tractability of the simulation.

The choice of GPU backend, such as JAX or Warp, directly affects how custom GPU-accelerated functions are implemented and optimised. Although both support parallelised numerical computation, their execution characteristics differ significantly. In this project, both JAX and Warp backends were implemented to demonstrate the flexibility of the simulation framework. However, due to the substantial performance advantage of Warp (as shown in Table 1.1), we primarily examine outputs generated using Warp. Its speed allows for quicker iteration and testing, making development hotfixes and parameter tuning considerably less time-consuming. We also select single-precision (FP32) arithmetic for the simulation domain to balance numerical accuracy with memory efficiency. Additionally, the standard D2Q9 lattice velocity model is used, common to 2D simulations and based on the assumption of equal Δx and Δy , as illustrated in Figure 1.2.

XLB requires the selection of a collision operator between two provided implementations, namely the BGK and KBC operators. As briefly covered in Chapter 1, the BGK model simplifies the complex collision term in the Boltzmann equation by assuming that the distribution function f relaxes towards a local equilibrium distribution f^{eq} at a rate governed by a single relaxation time τ . Assuming a normalised timestep $\Delta t = 1$, the discrete evolution of the particle distribution function in the Lattice Boltzmann Method is given by:

$$f_i(\mathbf{x} + \mathbf{c}_i \Delta t, t + \Delta t) = f_i(\mathbf{x}, t) - \frac{\Delta t}{\tau} (f_i(\mathbf{x}, t) - f_i^{\text{eq}}(\mathbf{x}, t)) \quad (2.1)$$

where f_i is the particle distribution function along direction i , $\mathbf{c}_i$ is the discrete lattice velocity vector, Δt is the timestep, and τ is the relaxation time controlling the rate of approach to equilibrium [26].

The relaxation time τ is linked to the fluid's kinematic viscosity ν through:

$$\tau = \frac{\nu}{c_s^2} + \frac{1}{2} \quad (2.2)$$

where c_s is the lattice speed of sound, with $c_s^2 = 1/3$ for D2Q9. To maintain consistency with lattice units, the physical kinematic viscosity must first be converted into lattice units using:

$$\nu_{\text{lattice}} = \nu_{\text{physical}} \cdot \frac{\Delta t}{\Delta x^2}$$

Only then can it be used to compute a valid τ value for simulation.

The relaxation time τ must be strictly greater than 0.5 to ensure numerical stability, as $\tau = 0.5$ implies zero viscosity and leads to instability through amplified numerical noise. Conversely, $\tau \gg 2$ leads to excessive diffusion and poor resolution of flow features. Notably, XLB expects the relaxation parameter in terms of $\Omega = 1/\tau$ [26], a form used to avoid repeated division and improve performance. In Newtonian flows, τ remains constant, but for non-Newtonian fluids, where viscosity ν varies with local shear rate $\dot{\gamma}$, a spatially and temporally varying $\tau(x, t)$ must be computed to reflect realistic fluid behaviour.

To create our rudimentary 2D cross section of an aneurysm, we must create a mask within the domain space to represent the walls, inlet, and outlet. Our domain geometry consists of 2 horizontal parallel lines with the centre of the top parallel section cut off and replaced with a semi-ovoid shape to create a representative bulge to replicate the aneurysm element of our simulation. We cap off the geometry to the left and right between the parallel lines to create what is essentially a rectangle with a bulge at the top. Our fluid flow would come in from the left and exit from the right of this shape. We can index this mask into the inlet (left-hand side of the geometry), outlet (right-hand side of the geometry), and the walls (parallel lines plus semi-ovoid shape) to create the overall testing domain. It is important to note that, to promote realism, a physical scaling factor between the lattice units and physical units Δx must be defined prior to creation to control attributes of the geometry such as the vessel diameter, and aneurysm width and height. Additionally, we want to minimise the domain such as to prevent unnecessary calculations that can be performed on elements outside the domain geometry.

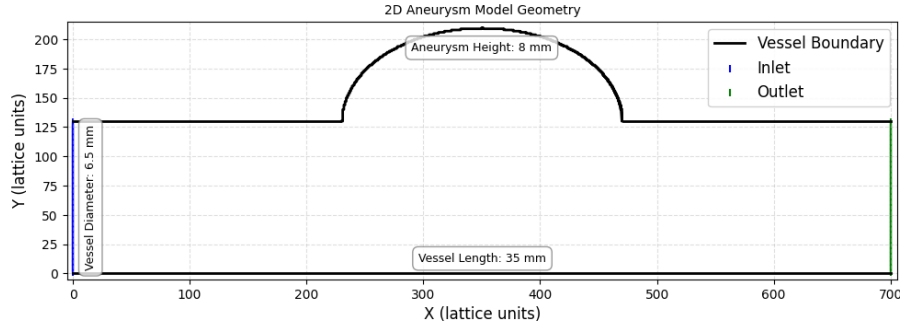


Figure 2.2: A construction of a 2D aneurysms with some features labelled. Fluid in the simulation is generated in the blue boundary, passes through the domain geometry, and exits out the green boundary.

Once the domain geometry has been set up, we can look to assigning the inlet, outlet and wall boundaries with BCs that govern the physiological conditions inside a blood vessel. XLB provides a variety of BC functions that can be applied to the different domain geometry indexes discussed above, described as functions that prescribe behaviours such as fixed inflow velocity, pressure outflow, or no-slip walls.

Each boundary function in XLB typically consists of two main parts: a condition that specifies when a cell is part of a boundary (e.g., based on its label or spatial location), and an action, which modifies the cell’s distribution functions or macroscopic variables (such as velocity or density) to enforce the boundary condition (BC) at each time step. This modular design allows users to define custom BCs while keeping the solver core general and efficient. For example, to simulate the inlet flow, a velocity boundary function can be assigned based on a time-dependent waveform, whereas wall boundaries can use bounce-back rules to enforce a no-slip condition. To improve flow stability and ensure realistic entry conditions, the vessel geometry is extended laterally of the aneurysm region. This allows the simulated fluid to stabilise before interacting with the aneurysm, minimising numerical artefacts. Once the BCs are defined, the Navier–Stokes equation (NSE) stepper function is selected to enforce the collision and BC operators, concluding the simulation setup phase.

In the simulation execution phase, a repeating loop is defined to apply the stepper function at every time step advancing the simulation by a small time increment Δt . At every time step, the stepper function updates the distribution functions and macroscopic quantities (such as velocity and density) across the domain based on the Lattice-Boltzmann dynamics described in eqn 2.1. we can also, though defining a post-processing interval that occurs every n loop iterations, extract the macroscopic properties of our simulation such as the velocity field and density distribution. We can export these values into files, such as .png snapshots or .vtk files, for detailed visualisation in external software such as Paraview. This loop continues until the required number of steps is reached.

The data extraction section can accommodate the possibility of implementable custom functions, such as the calculations for WSS shown in eqn 1.5. The .vtk file type can flexibly accommodate additional fields and metadata without disrupting the underlying mesh or flow

data. It is important to note that the data extraction process is far more expensive in terms of compute and memory requirements, so we are constrained by the GPU's VRAM, compromising on absolute precision and accuracy by limited Δx and Δt on local systems. Shared memory architectures, such as those available on HPC systems, Apple Mac systems, and the new Nvidia DGX Spark systems theoretically can support demanding simulation workloads due to their high memory capacity via the unified memory architecture.

2.3 Aims and Objectives

By developing a comprehensive analysis of the simulation space, we are able to create a set of aims and objectives that we need to achieve a successful project. We display the functional and non-functional requirements in the class diagram shown in Figure In general, we can identify 5 major objectives related to project success.

Our first objective would be to upgrade the generalised XLB flow to allow for modifiable vascular structures, an example shown in Figure 2.2. To do this, the simulation needs to incorporate Δx and Δt conversions to the variables passed into the simulation setup phase. Based on user-provided blood vessel and aneurysm parameters, our setup phase will dynamically create a tight computational domain, optimising resolution via experimentation while minimising unnecessary computational space.

We also need to correctly index the wall and outflow nodes shown in Figure 2.2. For the wall boundary condition, we opt for the provided fullway bounce-back scheme. While the halfway bounce-back method offers a more accurate representation of the wall location, potentially improving estimates of the velocity gradient and wall shear stress (WSS), it was found to cause fluid leakage and instability in our setup, likely due to mismatches between wall geometry and lattice alignment. In contrast, the fullway scheme provides a simpler and more stable alternative, especially in domains with curved or irregular geometry. Although it may introduce slightly higher error near walls, its robustness made it a reliable choice for our simulation framework. We also select the extrapolation outflow BC to allow the fluid to exit the domain with minimal artificial reflection, an unrealistic consequence when considering the continuation of blood flow in the vessel external to the lattice geometry. As per Figure 2.1, we still include the choice of lattice model (D2Q9), the GPU backend (Warp or JAX), floating-point precision policy (FP32), however we may choose to omit the omega variable to account for variable viscosity, discussed further in this section.

Our next objective is to replicate physiological blood flow by incorporating pulsatility at the inlet. Our overall goal for this section is to develop a custom modification of the provided Zou-He BC that can take a 64-point sample and perform linear interpolation between points to vary the inlet velocity accordingly. To do this, we require a fluid flow graph (which can be adapted from real or simulated data), digitised and normalised to a standard 64-point sample to ensure compatibility and smooth integration into the solver. We then derive the velocity profile at the inlet of our domain geometry and inject these results into our GPU backend using

4 statically defined 4x4 matrices. This optimises performance in the GPU architecture by using a standardised matrix in read-only format.

The third objective is to handle the non-Newtonian properties of blood. We start by introducing an approximation of this behaviour using the Carreau-Yasuda (CY) equation:

$$\mu(\dot{\gamma}) = \mu_{\infty} + (\mu_0 - \mu_{\infty}) [1 + (\lambda \dot{\gamma})^a]^{\frac{n-1}{a}} \quad (2.3)$$

Where $\mu(\dot{\gamma})$ is the dynamic viscosity at shear rate $\dot{\gamma}$, μ_0 is the zero-shear viscosity, μ_{∞} is the infinite-shear viscosity, λ is a time constant related to the transition between μ_0 and μ_{∞} , n is the power-law index describing the degree of shear-thinning, and a is a parameter controlling the sharpness of the transition. There are other models available to approximate non-Newtonian rheology, such as the power-law model, Casson model, and Cross model, however we select the Carreau-Yasuda model as it has been extensively validated and widely adopted in biomedical engineering for simulating blood flow

We begin to incorporate the non-Newtonian properties by first realising that variable viscosity leads to an embedding issue with existing collision models available in XLB. The reason is down to the method of calculating the relaxation time τ , which assumes a fixed kinematic viscosity ν . Varying viscosity, leads to varying relaxation times, which collision operators like BGK cannot support natively. This leads to an interesting dilemma. We can either implement a collision operator, such as the multiple relaxation time (MRT), capable of handling this varying viscosity, adapt the standard BGK collision operator to embed non-Newtonian effects, or add a new BC to directly influence the macroscopic and mesoscopic properties associated with non-Newtonian blood flow. In all cases, we must incorporate the CY equation as a native extension to the implementation. Overall, we choose to go for the non-Newtonian BGK method for its relative simplicity, compatibility with XLB's existing architecture, and ease of integration into the Python-based framework, allowing us to approximate shear-dependent viscosity effects by dynamically updating the relaxation time τ based on local shear rate using the Carreau-Yasuda (CY) model.

Our fourth objective is in relation to the execution and post-processing stage. We need to develop functions that can allow the simulation to run for a set amount of simulated time based off values of Δt . We also want to track key metrics such as simulation runtime, vessel dimensions, lattice grid dimensions, values of Δt and Δx , and the different combinations of selected flow properties. We also want to modify the post processing stage to incorporate and output WSS calculations to the .vtk files, bearing in mind the effects non-Newtonian viscosity. We can adapt the calculations to account for this using the following equation:

$$\tau_w = \mu(\dot{\gamma}) \left(\frac{\partial u}{\partial n} \right)_{\text{wall}} \quad (2.4)$$

which simply takes the formula for a consistent relaxation parameter shown in eqn ... and allows for variation of viscosity via the function $\mu(\dot{\gamma})$, implemented as the CY model in eqn.... We take the derived macroscopic results and assume that $\dot{\gamma} \approx \left| \frac{\partial u}{\partial n} \right|$, to then give the respective viscosity, subbing it into eqn 2.4. This assumption is valid because, near vessel walls, the flow

is dominated by shear deformation in the direction normal to the surface. Hence, the magnitude of the velocity gradient across the wall approximates the local shear rate sufficiently accurately for WSS estimation in laminar, non-Newtonian blood flow conditions.

To calculate the WSS under dynamic conditions, the implementation must estimate the wall normal, typically by approximating the local tangent using neighbouring points. Once obtained, the velocity from the nearest fluid node is used to compute the velocity gradient via a finite difference approximation. Since a fullway bounce-back boundary condition is employed, the physical wall aligns with the wall node itself. As such, the velocity gradient is estimated directly using $\frac{\partial u}{\partial n} \approx \frac{u_{\text{fluid}}}{\Delta x}$, where u_{fluid} is the velocity at the adjacent fluid node. This approximation avoids correction for any half-lattice offset and simplifies implementation, though it may introduce minor inaccuracies near curved or irregular boundaries.

A core objective of this project is to develop visualisation workflows that support validation of simulation outputs. ParaView is used as the primary post-processing tool to explore flow fields, detect vortices, and examine bulk fluid behaviour. Complementary to this, Jupyter notebooks are employed to validate physiological accuracy by comparing extracted features, such as inlet velocity, WSS, and pressure gradients, against expected ranges. Together, these methods provide qualitative and quantitative checks to ensure clinical realism.

2.4 Project Management, Version Control, & Testing

The project began under time constraints, which necessitated an agile and adaptive approach to planning and execution. Initial efforts focused on evaluating several lattice-Boltzmann frameworks, like XLB, LettuceCFD, and Palabos, to identify a platform capable of supporting GPU-accelerated blood flow simulation. The chosen system was intended to support future integration into clinical workflows, particularly within the NHS, while also serving the needs of academic researchers. As such, the selection criteria prioritised long-term usability, extensibility, and accessibility for non-technical users.

Due to the project's late start, no formal Gantt chart or structured project timeline was used. Instead, a lightweight agile methodology was followed, using two-week sprints with clearly defined short-term goals. Sprint tasks included preliminary research, such as evaluating suitable open-source libraries and understanding how lattice-Boltzmann simulations operate, followed by the creation of the simulation domain, and the development of core components including boundary condition handling and collision operators. This iterative process allowed development priorities to evolve alongside a growing understanding of the domain-specific challenges. While progress was not tracked using formal project management software, regular bi-weekly meetings with the project supervisor served as key checkpoints to evaluate outcomes, resolve blockers, and refine implementation direction.

Version control was managed using Git, with the full codebase hosted in a dedicated GitHub repository. This repository included a Docker-based environment built on Ubuntu 22.04 with

CUDA support for GPU acceleration. It contained executable scripts to install all dependencies, launch the simulation environment, and run blood flow models through either CLI flags or direct script execution. This setup was designed to ensure reproducibility and ease of deployment, particularly for future users unfamiliar with the development stack.

Development followed a branching model to isolate feature work, track milestone completion, and enable safe code integration. GitHub Issues were used to record bugs, feature requests, and implementation goals. Workflow testing was performed manually in a local GPU-enabled environment before merging new features into the main branch. This decision was driven by the need for GPU resources, which GitHub Actions does not support. Additionally, the simulation pipelines were too computationally intensive to be run reliably via automated CI workflows. Manual testing ensured simulation correctness and system stability without overloading remote runners or compromising reproducibility.

Testing itself was primarily qualitative and conducted throughout the development process. While automated unit testing was not implemented, the simulation outputs were manually verified through visualisation tools and compared to known or expected flow behaviours. This included assessing the accuracy of velocity profiles, the enforcement of boundary conditions, and general system stability. Although this approach was appropriate for the exploratory nature of the work, future improvements could include automated test cases and regression tests to support long-term maintenance and ensure numerical correctness across simulation runs.

Chapter 3

Implementation and Validation

This section details the implementation of the Lattice-Boltzmann method (LBM) for simulating pulsatile blood flow through a 2D aneurysm cross-section of a CCA using the XLB framework. It also outlines the integration of BCs, physiological flow data, and post-processing techniques for extracting key haemodynamic metrics such as velocity fields and wall shear stress (WSS), alongside validation against expected physical behaviour.

3.1 Implementation Details

We begin by discussing the implementation of our simulation domain, where we define functionality to create and configure a simulation domain with realistic physical parameters describing the aneurysm and vessel diameter and length characteristics, while handling the spatial and temporal resolution via programmatic assignment of Δt and Δx . It defines a 2D vessel with an aneurysm bulge in lattice units similar to Figure 2.2, finds the optimal computational grid size, and calculates crucial simulation parameters like the relaxation parameter (ω) based on blood's kinematic viscosity to account for using the standard provided collision operators. These values are then passed to the simulation class to then, similar to Figure 2.1, initialise and setup the simulation domain.

For the execution stage, we implement functionality to run the simulation domain for a specified amount of simulation time, allowing for a warm-up period (set to 1 second) to stabilise fluid flow. This custom "run for duration" function applies the temporal resolution to automatically derive the number of steps needed to replicate the given time. Via the definition of a frame-per-second (FPS) variable, we can derive the post-processing interval we want to use to record the simulation output by saving images, .vtk files, and by the end of the simulation, the parameters used and simulation performance, all saved to a folder which can be accessed within the local workspace. With this, we successfully complete objective 1 by allowing for flexible domain generation and boundary selection (mentioned in the Aims and Objectives section).

To create a custom velocity-varying inlet BC, we ask the user within the setup phase to select a velocity flow profile, which has been derived by the utilisation of a digitiser upon Figure 3.1 which gave 2 flow profiles for the CCA and Internal Carotid Artery (ICA). This particular flow model was chosen mainly as a possible solution (ref), showing the feasibility of our simulation with both graph and point-based inflow data. The user can select between these 2 profiles, however for this case we focus on the CCA profile as we input the standard params for a CCA's diameter. When the user select this, the program normalises the graph into 64 discrete points and preloads these into WARP as 4 constant 4x4 arrays to then be used within the Zou-He BC.

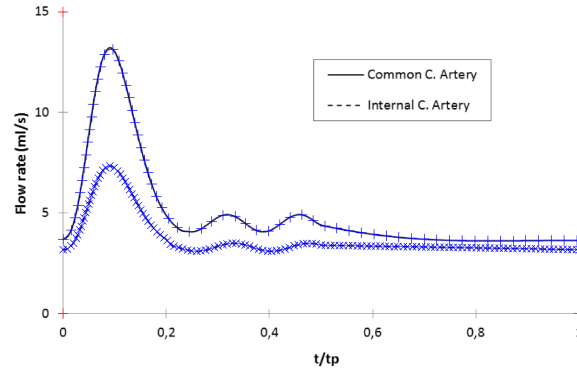


Figure 3.1: A digitised version of a simulation-derived flow model for the CCA and ICA. Crosses represent a point extracted from the flow graph

XLB’s implementation of the Zou-He BC is bugged. The timestamp is not connected up to the rest of the implementation, and therefore, we resort to overhauling the current implementation as a new BC to introduce the time dependency and 64 discretised points. This new BC preserves the core ZouHe BC mechanism, but allows for variable outputs by interpolating these 64 points according to the current timestep. This allows us to stay clear of a full overhaul of the XLB framework, as injecting these 64 points avoids the need to redesign the kernel and WARP functions, instead requiring little changes to the NSE stepper function, also overridden to allow for the timestamp functionality to work.

To satisfy objective 3, we implement another custom class of functions that work coherently with the BGK operator. Discussed in 2.1 Aims and Objectives, we calculate a local, shear-dependent viscosity based on the CY model shown by formula ... and Figure . This local shear rate is estimated by taking finite differences of velocity gradients since the full velocity field might not be accessible. We then convert the viscosity to an omega value, apply stability constraints in the range of 0.55 to 1.95, and apply this with the BGK model on a per lattice point basis.

For objective 4, the WSS is calculated within the post-processing stage. We implement this by an external utility function that accepts the wall indices as input. Following a similar approach to Objective 3, the function approximates local velocity gradients via finite difference methods. We convert these results into physical units via the lattice conversion factors Δx and Δt . To approximate the normal vector at each wall point, direction vectors from the wall towards adjacent fluid cells are accumulated and normalised. We can use these normals to calculate the components of the strain rate tensor based on the velocity gradients. The local shear rate is therefore derived, and the WSS is calculated via the implementation of eqn ... (2.4), with the CY model using the same parameters as with objective 3. We decompose the WSS into its x and y constituents and store this, plus the binary wall mask and WSS magnitude, to our VTK files. This process occurs every time the post-processing stage is called, operating within the designated FPS to avoid too much calculation.

Our final objective involves the creation and utilisation of basic visualisation tools to view the results in a comprehensible and reputable format. We use a combination of Paraview and

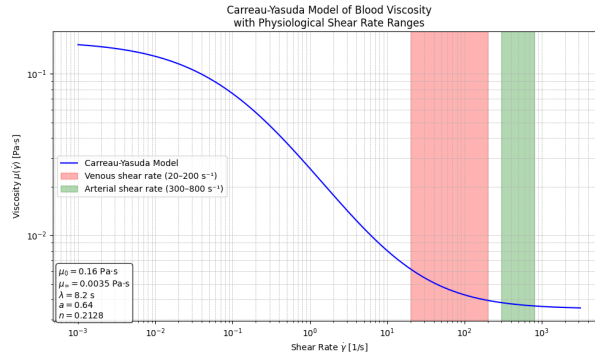


Figure 3.2: The CY model describing the relationship between shear rate and viscosity, with additional shear rate ranges for large vascular structures

Jupyter notebooks to play back the simulation. We use Paraview to provide a relatively fast playback of the simulation domain. The notebooks are used to view the velocity field, WSS, compare experimental and reference values, assess for divergence and curl, and validate properties such as consistent density and no leaks in the domain, which is crucial for simulation accuracy. To do this, we provide the path to the aneurysm simulation results directory to the visualisation tool (which can be more than one path depending on the functionality, like comparing outputs), unload the appropriate attributes from a selection of velocity and WSS's magnitude and x and y components, simulation density, and wall mask, We can then use graphing libraries, such as matplotlib, to provide a visual representation with colorbars and labelling as necessary, in an interactive environment.

3.2 Implementation Challenges

XLB as a library is currently an open source project run by researchers at Autodesk. The paper detailing its functional capabilities was released in 2024, outlining how the JAX framework is used to conduct LBM simulations accelerated by the GPU. Since then, XLB has remained in active development, adding features such as compatibility with the WARP backend, additional boundary conditions, and improvements to both usability and extensibility. With this, however, come some shortfalls.

It is critical to understand that users attempting to use and extend XLB require knowledge of its core components, code structure, and layout—alongside an understanding of the governing properties of the Navier-Stokes equations. All functionality inherits from a core class responsible for building the GPU backend. Focusing on the Warp backend, this includes kernel setup and activation, as well as the associated handler functions for different operations. Familiarity with these, in addition to the fundamentals of GPU parallelism, is necessary to implement custom collision operators, boundary conditions, and domain geometries.

At the time of writing, XLB does not offer widely available documentation or concrete guidance on implementing custom COs and BCs, particularly when working with the Warp backend. As a result, a significant amount of time must be spent examining XLB's source code

to understand class interoperability and internal behaviour, as well as understanding and learning the Warp backend. This differs notably from more established LBM simulators such as Lettuce CFD, which provide formal documentation and structured onboarding material, making extension and adoption far more intuitive.

Because XLB is manned by a small team, certain bugs exist when using some of its current features. Examples include the previously discussed de-linked timestamp for boundary conditions on the Warp backend, and the inability to extend the ZouHe BC with variable velocity profiles as a component of time. This required some radical thinking when attempting to implement a relatively simple boundary condition, as we aimed to avoid substantial changes to the function signature that would have introduced complexity across other classes such as the NSE stepper.

Implementing custom functionality to run alongside the simulation is not intuitive. When defining new boundary conditions or collision operators, we typically inherit from the `Operator` class - either directly or via subclasses such as `Stepper`, `Collision`, or `BoundaryCondition`. While the `Operator` framework provides a solid foundation with functional identifiers to manage application logic, it could benefit from the integration of a function factory to streamline the creation and registration of custom behaviours, making extensions more modular, readable, and developer-friendly.

3.3 Validation of Key Objectives

By the nature of our implementation and to conclude with our overall objective to produce a system that can accurately simulate fluid flow within a given 2D vascular structure, we must validate the domain and new implementations added to our solution. This includes an overview of objectives 1 through 4, as the developed visualisation tools are simple to construct and use. In each case, we validate by independently verifying each component, keeping things like the BCs and COs constant, varying if required. Our simulation methodology remains the same, but involves testing with different domain geometries to examine the effects of each implemented component.

To validate our implementation, we use a combination of visual inspection and mathematical reasoning. We begin by verifying whether the fluid flow is contained within the domain geometry and that no fluid leaks into the domain space. This involves checking for continuity and consistency across boundary interfaces. As outlined in the NSE equations in eqn ..., we want to assert the conservation of mass by finding the value of divergence to be adequately close to 0. We also look for signs of leakage or artefacts that would violate physical constraints. Visually, we confirm that no fluid escapes the domain, and computationally, we monitor the density field to ensure it remains constant throughout the simulation. A continuous density and volume imply conservation of mass, which supports our assumption of incompressible flow. We also want to ensure the simulation does not encounter numerical instability, which is detectable by the failure of the simulation to output non-numeric values tending towards infinity.

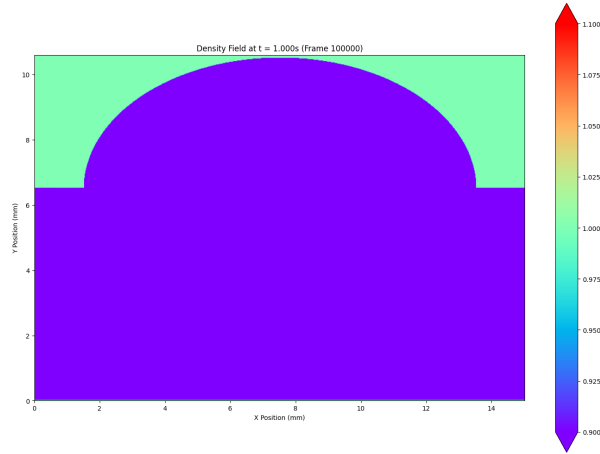


Figure 3.3: Density field at $t=1$. Shows the separation of distinct densities, indicating no fluid leakage

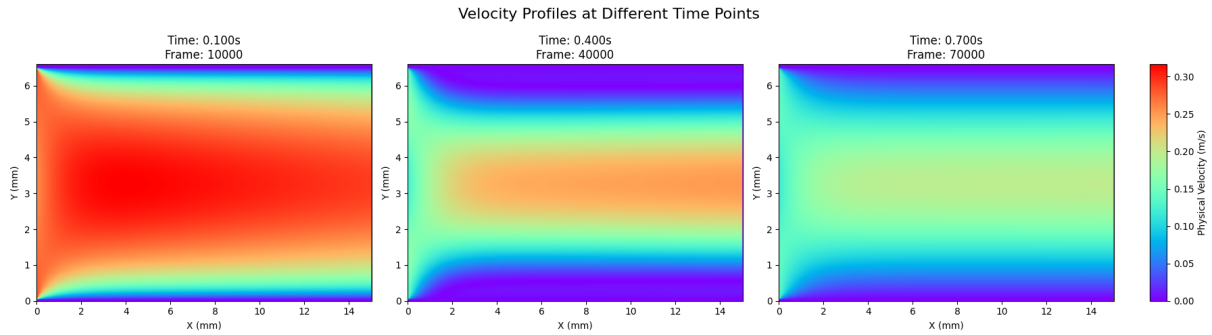


Figure 3.4: Pipe model with the time-dependent ZouHe BC at different times

Validating the second objective, we compare the results against a known analytical reference model. For this, we construct a 2D cross section of a constant-diameter pipe, using the standard BGK operator, and wall (fullway bounceback) and outlet (extrapolation outflow) BCs, while including the custom `TimeDependantZouHeBC`. We extract the simulation data as normal via the implemented post-processing methodology to focus on velocity magnitude and direction. We plot the prescribed inflow velocity profile against the simulated velocity at a corresponding cross-section over time. This allows us to visually assess how well the simulation reproduces the expected behaviour. We can confirm this component works by using a waveform correlation and normalised Root Mean Square Error (nRMSE) metrics between our fluid flow model and the maximum magnitude. A high correlation and low nRMSE indicate that the simulated velocity closely follows the analytical inflow profile, validating the correctness of the velocity propagation through the domain.

To validate the third objective, we first run a reference flow simulation through a straight tube using the BGK model with a fixed relaxation parameter ω corresponding to Newtonian fluid flow. We then repeat the simulation, replacing XLB's BGK operator with the custom non-Newtonian model. For both simulation cases, we construct the simulation environment by assigning a constant velocity to the ZouHe BC, and using extrapolation outflow for the outflow and fullway bounceback for the walls. We also run the simulation for a set warm-up period (2 seconds of simulation time) to allow the fluid flow to stabilise, and extract results when the

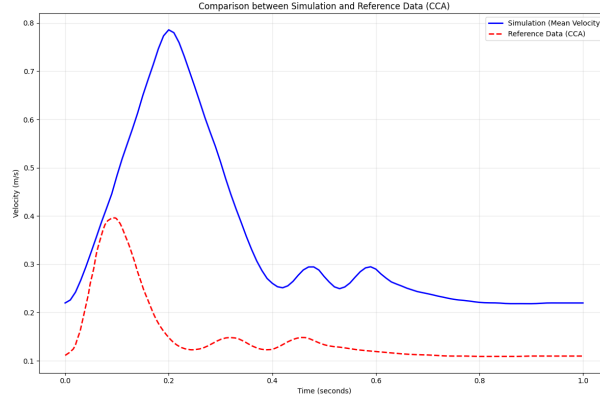


Figure 3.5: Comparison between simulation and reference data for the CCA flow profile

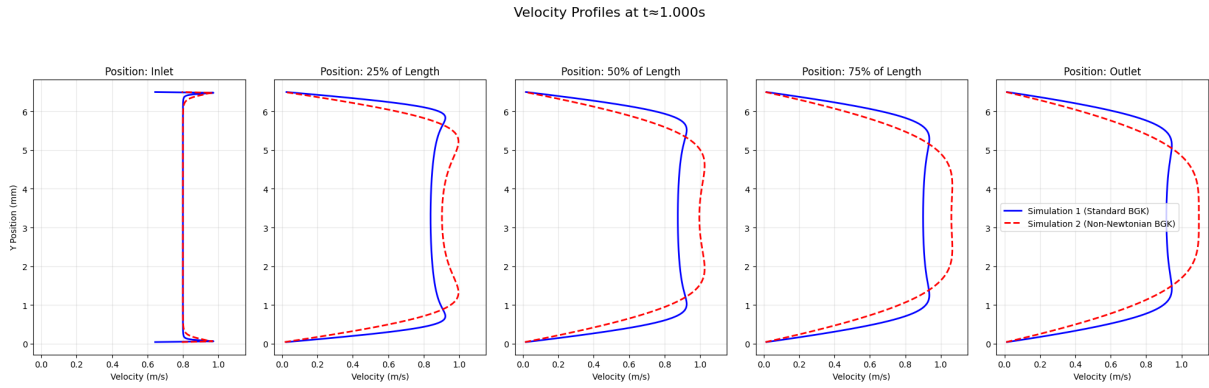


Figure 3.6: Velocity profiles at varying positions of the pipe, showing stabilising flow with length

warm-up period is complete. By comparing the velocity profiles near the vessel wall, we expect to observe a noticeable reduction in velocity in the non-Newtonian case due to shear-thinning effects. This behaviour is characteristic of blood flow and aligns with physiological expectations. To illustrate this, we generate visual heatmaps of the velocity field at steady-state, highlighting differences in flow distribution between the two models.

To validate the fourth objective, involving the calculation of WSS, we perform a comparative visual analysis between the aneurysm model and a straight reference vessel. By examining the spatial distribution of WSS, we identify regions of elevated shear, particularly focusing on the aneurysm neck and dome. When comparing, we expect high WSS along the aneurysm neck due to flow acceleration and constriction, while the bulb typically exhibits more complex, possibly

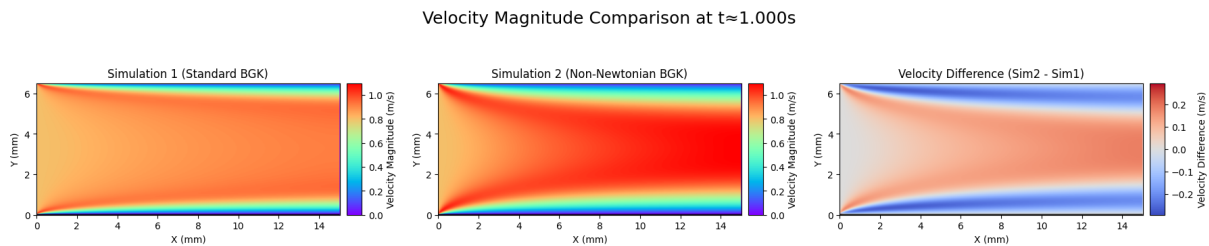


Figure 3.7: Comparison between the BGK and non-Newtonian BGK operators, with difference shown

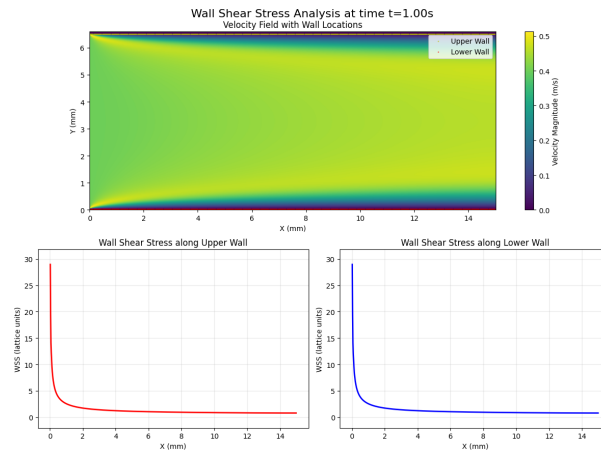


Figure 3.8: WSS for the standard BGK CO and ZouHe BC

lower or oscillatory shear patterns. Comparing fluid flow with the directional property of WSS at the tangent to the wall can verify the correct computation. These observations are interpreted in light of eqn ..., which defines WSS as proportional to the local velocity gradient at the wall. Additionally, we compare WSS outputs with and without the application of a variable viscosity model to assess the impact of shear-thinning behaviour. This highlights how non-Newtonian effects influence stress distribution and offers insights into physiological relevance and potential rupture risk.

Chapter 4

Results, Evaluation and Discussion

4.1 Evaluation

We begin the evaluation by observing the effect of our simulation components with the reference model before analysing the overall aneurysm component.

We start by looking at Figure 3.4, showing the pipe model that keeps the standard BGK CO and applies the custom time-dependent ZouHe inlet BC. We show times of 0.1, 0.4 and 0.7 to highlight some of the more significant phases of fluid flow, where we can see a decent correspondence between the velocity stream and the CCA graph provided by Figure 3.1.

Analysing this figure shows a jetstream influx at $t=0.1$, followed by a relaxation of fluid and reduction in inflow velocity over time, dispersing this energy across the wall boundaries. We expect this from our simulation given the fact that the viscosity tends to dissipate energy by resisting flow over a given time and limited fluid inflow at the latter stages, showing that the simulation temporally behaves as expected across the given timeframe.

We can analyse further the inflow component by taking the average velocity of the model and comparing it with the derived calculations of our preferred and set inlet velocity. Looking at Figure 3.5, we notice that our simulation data is significantly larger than the expected velocity model, and shifted to the right. We can explain this by interpreting the concepts of energy conservation. When observing Figure 3.4, between the ranges of 0-1 mm, we notice this cone-like shape at the inflow side, which is an effect of the time-dependent ZouHe BC applying a velocity across the entire inlet boundary. By an assumption made in the code, we reflect the inflow velocity to be exactly the velocity of the CCA profile. Taking the example at $t=0.1$, a spike of fluid corresponding to about 0.4 m/s is injected and translated to a calculated average of 0.8 m/s max peak velocity. This doubling in velocity is valid when considering the stabilisation of fluid flow across the domain, shown in Figure 3.6, where we see Poiseuille flow develop for both Newtonian and non-Newtonian fluid flow. We note that the Poiseuille flow is naturally parabolic, roughly fitting the notion of a doubling in velocity as the parabolic nature shows low to no velocity at the side, with most of the velocity concentrated at the centre. We therefore inject too high bloodflow in the simulation, causing unrealistic velocity projections. We also note the shift in waveform to the right by about 0.5 seconds in Figure 3.5. This is due to taking the mean velocity of the overall simulation as opposed to simply the average velocity at the inlet, which results in a phase shift due to temporal deviations in the velocity flow profile.

Moving on to modelling non-Newtonian blood flow, we examine Figures 3.6 and 3.7. Figure 3.7 shows the relative difference between the non-Newtonian and Newtonian solutions, from shows the differences between the 2 models. When looking, we notice the larger negative differences

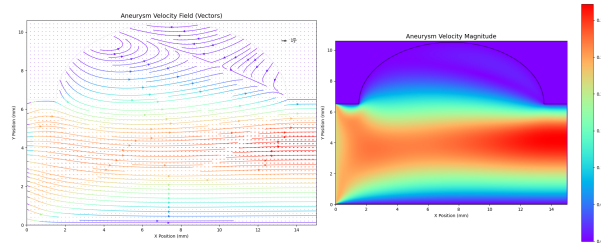


Figure 4.1: Aneurysm velocity field and magnitude at $t=0$, showing no flow in or out the defined walls

closer to the walls, blending out to become large positive differences when tending towards the centre of the pipe. This corresponds nicely with the expected result, where we observe the shear thinning behaviour (the slowdown) of blood close to the walls in correlation to eqn We also note the centre speed is much higher for the non-Newtonian BGK simulation, which we can attribute to the viscosity regime for arterial vessels outlined in Figure 3.2, compared to the larger static kinetic viscosity variable used in the standard BGK model. Looking at Figure 3.7, we can confirm the variations on velocity match what we expect, but critically, we do not see the full formation of our flow profile after 15mm. We expect this, given the fact that the flow profiles develop when given a significant time and, crucially, a significant pipe length. It is entirely possible to accelerate the development of this flow profile if the respective Poiseuille and blunted flow (parabolically shaped but with a flattened peak) can be mathematically implemented at the inlet, parallel to the fluid direction at the inlet.

Analysing the results for the WSS calculations, we take the straight pipe model and analyse the magnitude of the WSS. Looking at Figure 3.8 to assess a standard example using XLB's toolset, we see a significant spike in WSS values between 0-1 mm for both the top and bottom walls on the figure. This observation concurs with previously made statements regarding the stabilisation of the fluid flow regime. Looking at the velocity field, the velocity between 0-1 mm range is close to our specified inlet velocity of 0.4 m/s. High velocities tangential to the wall result in high WSS values as denoted by eqn As the flow progresses, the WSS decreases hyperbolically and stabilises at about 1 Pascal, which is what we would expect from steady-state fluid flow. Both upper and lower WSS readings correlate very closely with each other, displaying virtually no error.

With the analysis of the components out of the way, we move to analyse the performance of the overall simulation with the aneurysm test case. We start by highlighting Figures 3.3 and 4.1, both demonstrating the required leakproof domain construction with tight boundaries and no visible numerical instability. The aneurysm uses all the previously discussed mechanisms to apply physical constraints to the simulation for realism.

When observing the live aneurysm simulation results, we see that, similar to the test case in for objective 2, the flow varies according to the derived CCA profile. We see the greatest inflow velocity at $t=0.2$, which then reduces down to its local trough. When reaching $t=0.35$, we start to see a very limited amount of recirculating fluid flow, before the overall flow rides the bumps and settles at $t=0.7$ and beyond. Some key points noted with this simulation is that we see

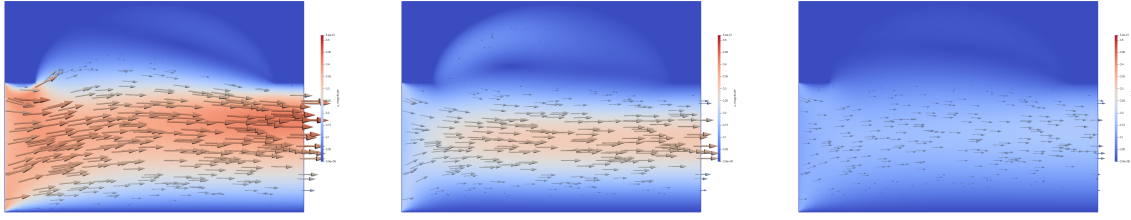


Figure 4.2: Paraview visualisation showing direction and magnitude of fluid flow at time $t=0.15$, 0.35 , and 0.7 respectively

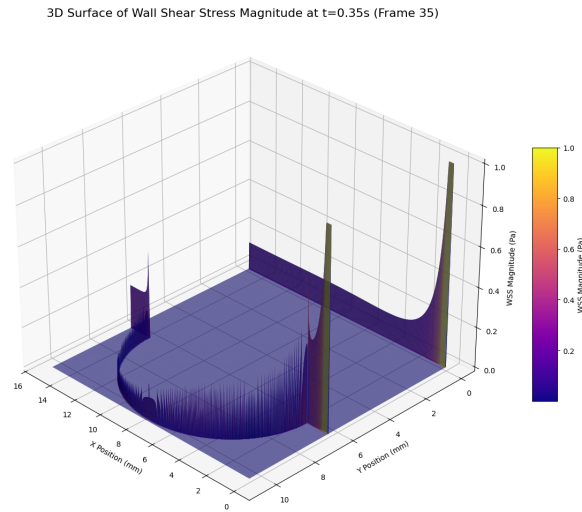


Figure 4.3: WSS magnitude of the aneurysm at time $t=0.35$

blood flow stagnation within the designed blood vessel, primarily within the aneurysm sac. As a result, we would expect the WSS within the sac to be extraordinarily low, which is indeed the case when examining figure 4.3. Typical ranges for healthy arterial vessels are between 1 and 7 Pa, but the majority surrounding the aneurysm is significantly below that. Vessels below 0.4 Pa can indicate possible endothelial dysfunction [33].

The performance metrics for our simulation hold up rather well. We test this system on an 11th Gen Intel(R) Core(TM) i7-11800H with clock speeds up to 2.30GHz, alongside an NVIDIA GeForce RTX 3060 Mobile with 6GBs VRAM. For a very high definition 2D cross section with $\Delta t = 1 \times 10^{-5}$ s, $\Delta x = 2 \times 10^{-5}$ m at 100 FPS, our total runtime was 1 minute 32 seconds, with 51 seconds dedicated to simulation time and 40 seconds put into post processing. The simulation maintained an average million lattice operations per second (MLUPS) of 2340.15, with peaks upto 5857.75 and lows (mainly coming from the initialisation stage) of 0.11. We note that typically, the compute cost went primarily to the post processing method, with an average post processing time of 0.4 seconds, compared to the average step time of 0.17 milliseconds.

4.2 Conclusions

Overall, the results from the simulation can be shown to validate the physical and computational accuracy of the proposed model and tools for a time-dependant, physiologically accurate blood flow through arterial structures, providing an example with the CCA. We successfully demonstrate that the custom inlet BC with given digitised point data can reproduce realistic velocity patterns over time, albeit with some limitations in velocity scaling and waveform phase alignment due to assumptions made during implementation.

Our comparison between Newtonian and non-Newtonian models highlights the significance of incorporating shear-thinning behaviour for arterial simulations, particularly in capturing more accurate velocity profiles and WSS distributions. Our non-Newtonian modelling captures improved central velocity and suppressed wall shear, aligning with the haemodynamic theory and enhancing the physiological realism of the model.

Wall shear stress (WSS) analysis further confirms the model's ability to capture key flow characteristics. In the straight pipe, WSS stabilised to expected physiological values ($\tilde{1}$ Pa), while the aneurysm test case revealed localised regions of low WSS (< 0.4 Pa), a condition associated with elevated risk for thrombosis and aneurysm progression.

Finally, the benchmarking on modern, albeit less performant, hardware shows strong performance metrics, achieving real-time simulation capabilities while optimising computational resources. Our implementation demonstrates a strong average of 2340 MLUPS, with peaks ranging above 5800 while sustaining high throughput. We identify that post-processing is our primary computational bottleneck, possibly down to I/O or CPU-bound processes.

In summary, our solution demonstrates the capability of delivering accurate, high-resolution haemodynamic analysis in clinically relevant scenarios. While we identify some fallbacks, mainly down to the lack of development for Poiseuille and/or blunted flow models to help develop realism, our solution lays down a strong foundation for future expansion into 3D models, advanced inlet profiles, and improved diagnostic capability through refined post-processing and machine learning.

4.3 Ideas for Future Work

Several promising avenues remain for extending the current implementation, particularly in preparation for deployment in clinical or research environments. This section outlines key areas for development, building upon the foundations established in this project.

4.3.1 Further Additions to the 2D Model

While the current 2D model captures key aspects of aneurysmal blood flow, it omits several physiologically relevant features. In particular, the exclusion of force terms such as f_{wall} and f_{body} from the Navier-Stokes formulation means that the model does not currently account for pressure-driven forces acting on vessel walls. These forces are important in understanding the mechanical stress and strain experienced by aneurysms, factors known to contribute to rupture risk. Further work could incorporate these terms, along with modelling external pressure events (e.g. sneezing or blunt trauma) that may cause sudden spikes in internal blood pressure.

Additionally, the model currently assumes rigid vessel walls. However, in vivo, blood vessels exhibit elastic behaviour, expanding and contracting in response to cardiac output. Aneurysms, in particular, may experience abnormal mechanical stress due to altered wall compliance. Future iterations could introduce wall deformability via a fluid–structure interaction (FSI) model, allowing the vessel boundary to respond dynamically to internal pressure. This would enable more realistic simulation of volume changes and the associated dampening of high-velocity inflow jets. Such extensions could also account for inter-patient variability, including genetically reduced vascular elasticity, altered material stiffness, and decreased recoil capacity.

To further improve physiological accuracy, a Womersley-type inflow boundary condition could be implemented to simulate pulsatile flow. This would allow for both positive and negative flow directions, better capturing the cyclical nature of arterial blood movement.

4.3.2 Transition to 3D Modelling

XLB defines classes that provide native support in both 2D and 3D simulations, and the current implementation can be readily extended to operate in three dimensions. Transitioning to 3D would enable more anatomically accurate studies and allow for direct modelling of complex geometries, such as bifurcations, stenoses, and irregular aneurysm sacs. While analytical benchmarks are less feasible in 3D, XLB supports simulation over STL mesh domains, allowing for direct use of medical imaging data.

This opens the possibility of simulating not only aneurysms but also related pathologies, including plaque formation, vessel narrowing, and drug or device interaction with local flow. However, 3D simulations come with significantly higher computational and memory demands. To address this, simulations could be run at lower spatial resolutions on hardware supporting unified memory architectures, such as NVIDIA’s DGX Spark systems with 128 GB of DDR5 RAM. Systems such as these must boast large memory capacity to accommodate large simulation domains, resulting in high-resolution simulations.

4.3.3 Extension of Warp to Distributed Architectures

While JAX supports ML and optimisation workflows, Warp remains better suited for high-performance fluid simulation due to its low-level GPU control and CUDA acceleration. A key direction for future development involves extending Warp to operate efficiently across distributed systems. This would enable large-scale simulations of anatomically complex vessels by leveraging multi-GPU or multi-node architectures through frameworks such as OpenMPI, which XLB currently provides experimental support for. A domain decomposition strategy could be used, segmenting the geometry into distinct parts, each processed by a separate node. For example, a vascular cylinder could be divided evenly across four GPUs, each responsible for computing a subsection of the vessel.

To maintain continuity between segments, a 3D stencil template would be defined at the interface of each subdomain. This allows computed outflow values from one segment to be transmitted and applied as inflow conditions to the adjacent segment. A slight overlap between neighbouring segments ensures physical consistency and stabilises the boundary data exchange. This method is particularly advantageous for irregular or sparse geometries, as it reduces computational overhead by eliminating whitespace (regions with no fluid activity) and maximising the effective simulation area per node. With appropriate load-balancing and partitioning algorithms, distributed Warp could support highly parallelised, anatomically realistic simulations in a cloud environment, providing a scalable foundation for clinical tools and research pipelines.

4.3.4 Incorporating ML Pipelines

Once 3D simulation capability is in place, machine learning pipelines can be introduced to extract predictive features from simulation output. Potential applications include aneurysm rupture risk prediction based on wall shear stress, pressure gradients, and strain profiles. Databases such as Aneurisk contain high-quality anatomical data which, when combined with simulated haemodynamic parameters, could support supervised learning tasks.

Additionally, machine learning could assist with preprocessing tasks such as velocity profile estimation at the inlet—an area that remains a key challenge in achieving physiologically accurate simulations. More advanced approaches, such as physics-informed neural networks (PINNs), could be explored to reduce computational overheads by learning approximations of the governing equations.

4.3.5 Implementing Clinical and Research-Based Tools

A complete pipeline suitable for clinical or research use would involve several stages: anatomical model generation (e.g. from 3D/4D MRI or CAD), simulation preparation (e.g. leak checks, boundary labelling), execution of the simulation, post-processing of haemodynamic features, and output to a review system. In the clinical setting, such a system must prioritise

interpretability, reliability, and integration into existing diagnostic workflows.

Deployment within the NHS would require extensive trialling. A phased approach could begin with local deployments within a specific trust, gathering feedback and refining the system before wider rollout. Radiologists equipped with high-resolution scanning technology could act as intermediaries, preparing the model and interpreting the results for surgical teams. Ultimately, this would support patient-specific modelling, enabling tailored treatment decisions and improved prognostic capabilities.

Pipeline: A model is derived or constructed using 3D/4D MRI scans (possibly using AI to label certain anatomy for the simulation to extract) or CAD software. Model is scanned for leaks in the system before being sent off for simulation. Simulation runs, stable fluid flow is developed after cooking in the sim. Key features extracted, sim returns. Depending on the nature of the sim, may do some extra stuff like running results through an ML pipeline to check for rupture risk on an aneurysm. Simulation is returned to whoever sent it. Radiologists and clinicians can review the data to provide further insights into treatment action and prognosis.

Development of this may require databasing the behaviour of every anatomical structure known. Predictive inlet velocity is a key issue. Without knowing this, simulation results are ultimately inaccurate. Also needs to be patient-specific, so getting as much vascular information as possible before leading up to the area of interest helps to establish highly stable and realistic blood flow, increasing accuracy.

Data needs to be returned to a high-performance playback system that is intuitive, easy to understand and handle, and provides as much detail as needed by clinicians.

Entire system needs trialling, especially for deployment within the NHS. Can focus locally for a specific trust before expanding out for wider adoption.

Can propose an integration pipeline. Look to the radiologist with capable 3D/4D MRI scanning technology. They can handle the I/O/P of this data, interpret it and describe to the doctors what they see and how to interpret the results. These conversations normally happen in teams if surgery is likely, so information can be easily distributed.

References

- [1] National Library of Medicine. Aneurysms.
- [2] Jie Liu, Xuan Zou, Yan Zhao, Zhangning Jin, Jun Tu, Xianjia Ning, Jidong Li, Xinyu Yang, and Jinghua Wang. Prevalence and Risk Factors for Unruptured Intracranial Aneurysms in the Population at High Risk for Aneurysm in the Rural Areas of Tianjin. *Frontiers in Neurology*, 13, 3 2022.
- [3] Ronil V. Chandra, Julian Maingard, Lee-Anne Slater, Nicholas K. Cheung, Leon T. Lai, Seana L. Gall, Amanda G. Thrift, and Thanh G. Phan. A Meta-Analysis of Rupture Risk for Intracranial Aneurysms 10 mm or Less in Size Selected for Conservative Management Without Repair. *Frontiers in Neurology*, 12, 2 2022.
- [4] H. Meng, V.M. Tutino, J. Xiang, and A. Siddiqui. High WSS or Low WSS? Complex Interactions of Hemodynamics with Intracranial Aneurysm Initiation, Growth, and Rupture: Toward a Unifying Hypothesis. *American Journal of Neuroradiology*, 35(7):1254–1262, 4 2013.
- [5] J R Cebal, F Mut, J Weir, and C M Putman. Association of Hemodynamic Characteristics and Cerebral Aneurysm Rupture. *American Journal of Neuroradiology*, 32(2):264–270, 11 2010.
- [6] Jordan King and David R. Lowery. Physiology, Cardiac Output, 7 2023.
- [7] Brian M. Howard, Ranliang Hu, Jack W. Barrow, and Daniel L. Barrow. Comprehensive review of imaging of intracranial aneurysms and angiographically negative subarachnoid hemorrhage. *Neurosurgical FOCUS*, 47(6):E20, 12 2019.
- [8] Fujimaro Ishida, Masanori Tsuji, Satoru Tanioka, Katsuhiko Tanaka, Shinichi Yoshimura, and Hidenori Suzuki. Computational Fluid Dynamics for Cerebral Aneurysms in Clinical Settings. *Acta neurochirurgica. Supplementum*, pages 27–32, 1 2021.
- [9] Hanieh Mirzaee, Thomas Henn, Mathias J. Krause, Leonid Goubergrits, Christian Schumann, Mathias Neugebauer, Titus Kuehne, Tobias Preusser, and Anja Hennemuth. MRI-based computational hemodynamics in patients with aortic coarctation using the lattice Boltzmann methods: Clinical validation study. *Journal of Magnetic Resonance Imaging*, 45(1):139–146, 7 2016.
- [10] Swetha Yogeswaran and Fei Liu. Vascular flow simulations using SimVascular and OpenFOAM. *medRxiv (Cold Spring Harbor Laboratory)*, 9 2021.
- [11] S P Kopysov, A K Novikov, N S NedoZhogin, and V N Rychkov. Scalability of parallel finite element algorithms on multi-core platforms. *IOP Conference Series Materials Science and Engineering*, 158:012055, 11 2016.

- [12] Florian Schornbaum and Ulrich Rde. Massively Parallel Algorithms for the Lattice Boltzmann Method on NonUniform Grids. *SIAM Journal on Scientific Computing*, 38(2):C96–C126, 1 2016.
- [13] Lukas Failer, Piotr Minakowski, and Thomas Richter. On the Impact of Fluid Structure Interaction in Blood Flow Simulations. *Vietnam Journal of Mathematics*, 49(1):169–187, 1 2021.
- [14] Haifeng Wang, Timm Krger, and Fathollah Varnik. Effects of size and elasticity on the relation between flow velocity and wall shear stress in side-wall aneurysms: A lattice Boltzmann-based computer simulation study. *PLoS ONE*, 15(1):e0227770, 1 2020.
- [15] Rossella Campobasso, Francesca Condemi, Magalie Viallon, Pierre Croisille, Salvatore Campisi, and Stphane Avril. Evaluation of peak wall stress in an ascending thoracic aortic aneurysm using FSI simulations: effects of aortic stiffness and peripheral resistance. *Abstract*, 2021.
- [16] John Hinch. *Fluid Dynamics II (D)*. Draft 1 edition, 9 2020.
- [17] Bastian E. Rapp. *Vector Calculus*. 12 2016.
- [18] Taha Sochi. Non-Newtonian Rheology in Blood Circulation. Technical report, 6 2014.
- [19] Eduardo Soudah, Eugenio Oate, and Miguel Cervera. (PDF) Computational Fluid Dynamics Indicators to Improve Cardiovascular Pathologies Diagnosis, 3 2017.
- [20] Vivig Shantha Kumar and Vignarth Shantha Kumar. High Wall Shear Incites Cerebral Aneurysm Formation and Low Wall Shear Stress Propagates Cerebral Aneurysm Growth. *Shantha Kumar / Journal of Neurology Research*, 8 2023.
- [21] A. Sommerfeld. *~Einæ Beitrag zur hydrodynamischen Erklrung der turbulenten Flssigkeitsbewegungen*. 1 1909.
- [22] Hack-Lyoung Kim. Arterial stiffness and hypertension. *Clinical Hypertension*, 29(1), 12 2023.
- [23] Andrew M. Jersey and David M. Foster. Cerebral Aneurysm, 4 2023.
- [24] Tristan Maquart, Romain Nol, Guy Courbebaisse, and Laurent Navarro. Toward a lattice Boltzmann Method for Solids—Application to static equilibrium of isotropic Materials. *Applied Sciences*, 12(9):4627, 5 2022.
- [25] Frank Weichert, Lars Walczak, Denis Fisseler, Tobias Opfermann, Mudassar Razzaq, Raphael Mnster, Stefan Turek, Iris Grunwald, Christian Roth, Christian Veith, and Mathias Wagner. Simulation of Intra-Aneurysmal Blood Flow by Different Numerical Methods. *Computational and Mathematical Methods in Medicine*, 2013:1–10, 1 2013.
- [26] Mohammadmehdi Ataei and Hesam Salehipour. XLB: A differentiable massively parallel lattice Boltzmann library in Python. *Computer Physics Communications*, 300:109187, 3 2024.

- [27] OpenLB. OpenLB 8211; Open Source Lattice Boltzmann Code 8226; OpenLB - Open source lattice Boltzmann code, 1 2025.
- [28] Z. Shi, G.Z. Chen, L. Mao, X.L. Li, C.S. Zhou, S. Xia, Y.X. Zhang, B. Zhang, B. Hu, G.M. Lu, and L.J. Zhang. Machine Learning–Based Prediction of Small Intracranial Aneurysm Rupture Status Using CTA-Derived Hemodynamics: A Multicenter Study. *American Journal of Neuroradiology*, 42(4):648–654, 3 2021.
- [29] British Medical Association. BMA and Secretary of State for Health discuss abuse crisis in the NHS and urgent need for reduction in bureaucracy for GPs - BMA media centre - BMA, 9 2021.
- [30] Jianping Xiang, Nicole Varble, Jason M. Davies, Ansaar T. Rai, Kenichi Kono, Shin-Ichiro Sugiyama, Mandy J. Binning, Rabih G. Tawk, Hoon Choi, Andrew J. Ringer, Kenneth V. Snyder, Elad I. Levy, L. Nelson Hopkins, Adnan H. Siddiqui, and Hui Meng. Initial Clinical Experience with AView—A Clinical Computational Platform for Intracranial Aneurysm Morphology, Hemodynamics, and Treatment Management. *World Neurosurgery*, 108:534–542, 9 2017.
- [31] Xiaofei Ren, Zheng Xin, and Feifei Liu. A non-Newtonian thermal lattice Boltzmann method for simulation of Rayleigh–Bénard convection of power-law fluids. *AIP Advances*, 13(11), 11 2023.
- [32] Fergus Shone, Nishant Ravikumar, Toni Lassila, White Rose Research Online, Fergus Shone, Nishant Ravikumar, Toni Lassila, Michael MacRaid, Yongxing Wang, Zeike A Taylor, Peter Jimack, Erica Dall’Armellina, and Alejandro F Frangi. Deep Physics-Informed Super-Resolution of Cardiac 4D-Flow MRI. Technical report, 2023.
- [33] Nikhilesh Bappoo, Maaz B.J. Syed, Georgia Khinsoe, Lachlan J. Kelsey, Rachael O. Forsythe, Janet T. Powell, Peter R. Hoskins, Olivia M.B. McBride, Paul E. Norman, Shirley Jansen, David E. Newby, and Barry J. Doyle. Low Shear Stress at Baseline Predicts Expansion and Aneurysm-Related Events in Patients With Abdominal Aortic Aneurysm. *Circulation Cardiovascular Imaging*, 14(12):1112–1121, 12 2021.

Appendix A

Self-appraisal

A.1 Critical self-evaluation

project:

horrible. I hate the state I have to submit this in. I was aiming for high marks at least in the 80s band. cannot further develop until after exams.

Lost essentially the entirety of semester 1 down to recurring illness and unplanned emergency surgery, resulting in 1 month off and another 1-2 months attempting to cover modules.

iterating through the solution I've seen improvements i need to make. things like developing an aneurysm sac mask to refine results for the WSS so i can focus more on that particular component. Love the detail, hate leaving it out

format of report is ok. definitely not my best work (yet). I've done demanding research projects before (EPQ). Everything about the report and its structure is not sitting right with me. mainly down to significant refinements and refactoring. Because of low time and the current risk of accidental plagiarism (need to still cite sources), I have to spell out these paragraphs as if they were in note form. would normally refine these later. mainly doing this to try and hunt marks, but not sure how effective this would be, given that the mark scheme mentions that everything must be brilliant to get the highest marks. always worth a shot to try something to hunt for marks, just in case the resit falls through.

The project code can definitely be improved. Repo is currently very cluttered with failed experiments and tests to try and run visualisers, good ideas to try and promote modularity. Want to make this an attractive repository to look at and work with. README is in a jumble as well.

Some good stuff. Project outcomes came through despite the whole refinement issue. gained knowledge on an ambitious, technically challenging and complex task. originally wanted to be extremely bold with the project (supervisor can attest to this), but had to scale back given the whole illness issue and having to start from scratch with the new idea in the beginning of semester 2.

Want to acknowledge the key role AI has played in this report. using GPT, Gemini, Deepseek for research purposes (with credible sources of course), fleshing out ideas, confirming thought processes, refining report structure and content in the context of the mark scheme, while using the assumption that AI systems can be treated as a really smart database. Coding aspects

primarily involved GitHub Copilot in ask, edit and agent modes, combined with Claude 3.7 and 3.7 thinking. Use of AI here was mainly for debugging, refining graphs and structure of notebooks, minor edits here and there (like adding certain flags for certain operators, which I know I can do, but just let the AI do it to avoid wasting time typing and debugging).

still a bunch more stuff to do. cross referencing, tying up loose ends in the report content, content and code refinement

A.2 Personal reflection and lessons learned

Start early. Did this with the EPQ too. Starting early allows you to think more about your project. So much better when you are just not busy and are thinking about what component of this task I need to do.

compared to the EPQ, this feels more restrictive. have to follow a set criteria and marking scheme, cant get supervisor to read the entire thing (for good reason, but becomes difficult to try and explain the problem of flow e.g.). Whereas the EPQ involved more critical thinking and philosophical argument (my EPQ was based on "Emulating consciousness using AI"). This was made pre-GPT

Wanted to do something more technical and challenging because I took the degree to enable me to go into whatever field I want to go in. Doing something diverse and technically challenging with many different physics and biological principles involved was particularly appealing.

lessons:

Time management in the face of adversity

hard to convey complex topics and subject domains concisely without losing out on key details

A.3 Legal, social, ethical and professional issues

While the current implementation remains a prototype with no direct clinical deployment, legal, social, ethical, and professional considerations become increasingly relevant when examining the system's future applications, as outlined in the Future Work section.

A.3.1 Legal issues

A key legal area to implementing LBM simulation technology into existing healthcare systems covers the compliance with data protection regulations, particularly the General Data Protection Regulation (GDPR). Patient data is regarded as sensitive, and as such, any transfer, processing, or storage of such information must adhere strictly to principles of data

minimisation and anonymisation. we can maintain GDPR compliance by pseudonymity required for the simulation and AI tools to work, which can include medical imaging or patient metadata. This typically involves removing direct identifiers (e.g. names, NHS numbers) and converting potentially traceable variables (e.g. date of birth to age) to prevent re-identification. In practice, this means the simulation would operate only on anatomical mesh representations and generalised physiological parameters, which are insufficient for identifying individual patients. Furthermore, any use of these data to train AI models must ensure that even inferred patterns cannot be linked back to identifiable subjects, requiring robust de-identification protocols throughout the pipeline.

Another key legal area surrounds the approval of medical devices in compliance with relevant medical device regulations. In the UK, systems used to inform diagnosis, treatment, or monitoring can be classified as a medical device under the UK Medical Device Regulations 2002 (as amended post-Brexit). As a result, the technology falls into a strict category requiring the demonstration on safety, effectiveness, and clinical validation. Our LBM simulator and related tools would need to undergo a conformity assessment and may require CE or UKCA marking, depending on the intended use and market. These regulatory frameworks ensure that such tools are subject to appropriate oversight and that their clinical claims are backed by evidence. Therefore, before such technology can be deployed in practice, it must navigate a rigorous legal landscape concerning both data governance and medical safety standards.

A.3.2 Social issues

The adoption of LBM simulation technology within the NHS must be viewed not only through a technological or clinical lens, but also with regard to its wider social impact, particularly how it aligns with current pressures on the UK healthcare system and public sentiment. For this, we must consider how this technology fits into national healthcare resource allocation decisions. The NHS faces intense scrutiny due to constrained budgets and long waiting lists. Introducing resource-intensive operations and solutions like the proposed LBM simulation allocated to a distributed architecture raises critical questions around cost-effectiveness. A thorough analysis must be conducted to assess whether the proposed clinical benefits, such as more precise modelling of blood flow or earlier detection of cardiovascular risks, can justify the additional costs when compared to alternative investments. Without careful training and procedural guidance, there is a potential for overdiagnosis or increased follow-up procedures for anomalous simulation results, which also possesses the capability to increase the workload on the already strained system. We believe we can clearly justify the need for such tools, given the additional insight. We cite the cost-effectiveness of prevention over treatment, benefits for enhanced research, exploring different treatment options, expanding capability into other AI-assisted systems as a cloud-based service, and reducing misdiagnosis.

Another factor involves the integration of LBM technology with existing digital infrastructure. An Electronic Health Record (EHR) is a digital version of a patient’s medical history that is maintained over time by healthcare providers, which can include patient demographics, medical

history, and laboratory and imaging results. The NHS operates on the basis of different trusts and institutions, and as a consequence, operates with different EHR platforms (e.g., SystmOne, EMIS, Cerner) joined onto a unified databasing system known as the NHS Spine. Therefore, for LBM simulation outputs to be clinically useful, they must be compatible with a variety of EHR environments. A key technical and social challenge is whether such integration can also connect with the NHS Spine, which supports key services like Summary Care Records and electronic prescriptions. Ensuring compatibility across this fragmented ecosystem is essential to making the technology accessible to clinicians across different trusts and regions, thereby avoiding digital exclusion and ensuring equitable benefit.

Some considerations must be given to clinician readiness and digital literacy. The varying skill set in digital literacy may result in additional specialist training required for those whom lack the necessary training or confidence to interpret complex simulation results. a lack of appropriate training and user-friendly interfaces can result in under utilisation and misinterpretation. This can existing inequalities between trusts that have invested more in digital transformation and those that have not.

Additionally, the ferocity behind the public and political debate around NHS spending adds another layer of complexity for implementation. Distributed systems, especially those relying on high-performance computing, are inherently energy-intensive and costly to maintain. With politics and top-level management currently oriented to restructuring the NHS to reduce overhead and eliminate costs, deploying such technology must be supported by strong justifications regarding its impact and scope. We can create enhance its perceived value by presenting the collection of LBM tools as a scalable solution that addresses a broad range of cardiovascular conditions rather than a niche research tool. positioning the solution as an enabler of more personalised, preventive healthcare, we can create a persuasive argument to justify its inclusion in long-term NHS digital strategy.

A.3.3 Ethical issues

One issue is the requirement for informed consent in the context of further testing, data gathering, and refinement of AI models. A system needs to be put in place to make patients aware of the anatomical structure of the compromised vascular region and suitable patient metadata that is required to create, develop and refine the AI models we can use. This includes understanding what the data will be used for, how it will be stored, and whether it will feed into AI development or validation. We can take measures to inform the processes we employ and to what standards we require to comply with GDPR, such as pseudonymising patient data, to allow the patient to make an informed decision, making it clear that patients have a clear and accessible ability to opt in or out of such data usage without it affecting the care they receive. Ensuring autonomy in this process is essential, particularly when AI systems may continue to evolve using real-world clinical data.

Another key ethical issue revolves around the transparency and explainability of AI systems when integrating LBM-derived features as input. Taking the aneurysm rupture predictor model

as an example, we are aware biomechanically that both very high and low WSS figures and high pressure acting on the aneurysm wall can lead to an elevated risk of rupture, but how can we conclude the AI system picks this up, and with several other factors like age and aneurysm size, what is more predominant in determining rupture risk. Such AI systems risk becoming “black boxes,” where neither clinicians nor patients can meaningfully interpret the reasoning behind a prediction. The decision must be traceable to understandable inputs, otherwise it risks the patient and clinicians trust, being unable to justify or critically evaluate the AI’s conclusions. Such issues may undermine the clinical accuracy and patient trust, making explainability a core ethical requirement for any AI-driven diagnostic or decision support tool. We can aim to reduce this by incorporating AI into the visualisation tool, showing the features it determines are significant indicators of rupture.

A.3.4 Professional issues

A core concern regarding professional standards is the preservation of clinical judgment in the face of simulation and AI systems. While the LBM toolset can offer high-quality haemodynamic insights alongside the AI assets that highlight simulation outcomes, such tools cannot be replaced for the clinical expertise of trained professionals. We must ensure that these systems are used as collaborative tools to augment decision-making rather than supplanting it. As part of training, we must make the principles and risks of automation bias profoundly known to those using these tools, encouraging critical judgment over blind recognition in the face of contradictory or misleading information. Practitioners must retain a holistic view of each patient, using simulation outputs as a supplementary layer rather than a definitive answer.

This naturally leads to the issue of upskilling the clinicians involved to ensure they understand not only how to operate these tools but also how to critically assess and interpret their outputs. We mention this within the social context; however, in this section, we want to address how such training is executed. Education programs, CPD (Continuing Professional Development) modules, and integration into clinical training will be required to embed a baseline technological fluency across specialisms. Such programs are able to bolster confidence in the reliability and limitations of these systems in order to use them effectively and ethically.

We can extend the concerns towards the data management and integration within the existing NHS infrastructure. LBM simulation can produce significant quantities of data when the simulation has been executed. As a result, there would be a demand to efficiently store the anatomical meshes and LBM simulation results, complying with NHS data governance protocols. For clinicians wanting to save the result of the simulation to compare after a given time, clear guidelines must be established regarding whether data is tied directly to identifiable patient records or stored in a segmented fashion to preserve confidentiality. This raises questions about whether raw meshes, processed results, or only key parameters should be retained, and how such data can be professionally and securely embedded within NHS server systems without introducing vulnerabilities or excessive strain on infrastructure.

Finally, system integration is a major professional consideration. To consider the idea of

integrating the existing NHS ecosystem, LBM-derived data must fit within existing EHR systems, ideally linking to the NHS Spine where appropriate. as mentioned in the social context of EHR systems, the diversity of these EHR solutions across different NHS trusts presents a difficult challenge to overcome. Professional developers and engineers must ensure compatibility and interoperability while avoiding independently operating systems that cannot share information with other systems - an independant silo - that can significantly slow down care, create errors, or reduce the impact of technological innovations. Only with robust professional planning, training, and data infrastructure can the true potential of LBM and AI tools be realised in supporting clinicians across the UK healthcare system.

don't want this technology to replace the expert opinion of doctors, but want to show the importance of these tools as a measure of collaborative effort between doctors and technology.

vvv **Maintaining Clinical Judgment:**

- **Issue:** Ensuring that technology aids, rather than replaces, the critical thinking and holistic judgment of healthcare professionals.
- **Example:** Professionals should be wary of "automation bias" – the tendency to over-rely on automated systems and ignore contradictory evidence.

need to educate people on how this tech works. need to see responses and how to integrate within the existing structure of the NHS.

resources derived from the anatomical model can be massive in file size. how do we store this data professionally to the existing NHS servers without compromising on patient confidentiality. do we store a mesh, the whole result for comparison after some time? can it be saved with the patient's details or does there need to be some kind of segmentation within the NHS's databasing system

Appendix B

External Material

<This appendix should provide a brief record of materials used in the solution that are not the student's own work. Such materials might be pieces of codes made available from a research group/company or from the internet, datasets prepared by external users or any preliminary materials/drafts/notes provided by a supervisor. It should be clear what was used as ready-made components and what was developed as part of the project. This appendix should be included even if no external materials were used, in which case a statement to that effect is all that is required.>